

Architecture

CRM/ERP Platform Documentation

System Architecture

Overview

The platform is a **metadata-driven multi-tenant CRM/ERP**. Instead of hard-coded tables for contacts, leads, etc., it stores everything in a generic **records** table where structure is defined by admin-configured **DynamicModule** and **Field** rows. This lets organizations create custom data models without code changes.



Frontend Architecture

Next.js App Router

Pages live under `mycrm/app/`. Route groups control layout:

Route group	Layout	Purpose
<code>(dashboard)</code>	Sidebar + topbar	Authenticated CRM views
<code>cf/</code>	Portal layout	Customer-facing portal
<code>f/[token]/</code>	Minimal	Public form submission
<code>(auth)/</code>	Centered	Login / register

Proxy Configuration

`mycrm/next.config.ts` rewrites `/api/v1/*` to `http://localhost:4000/api/v1/*`. The frontend never hardcodes the backend URL — it always calls `/api/v1/...`.

State Management (Zustand)

13 stores in `mycrm/store/`:

Store	Purpose
<code>auth.store.ts</code>	Current user, token, org
<code>blueprint-runtime.store.ts</code>	Active blueprint state
<code>module.store.ts</code>	Loaded module definitions
<code>record.store.ts</code>	Active record data
<code>notifications.store.ts</code>	Real-time notifications
<code>workspace.store.ts</code>	Tasks/notes sidebar
<code>ui.store.ts</code>	Sidebar state, modals
+ 6 more	Feature-specific

Backend Architecture

NestJS Modules

Each feature is a NestJS module with `Controller → Service → Prisma` layering. 46+ modules total (see [API.md](#)).

Global modules (available everywhere without importing):

- `PrismaModule` — database client
- `ConfigModule` — environment variables
- `ThrottlerModule` — rate limiting (100 req / 60 s)

Multi-tenancy

Every request carries a JWT with `organizationId`. All Prisma queries include `where: { organizationId }`. The `SUPER_ADMIN` role can query across organizations.

Permission Resolution

```
SUPER_ADMIN → bypass all checks
↓
Role-level permission (Permission table)
↓
Department override (departments.permissions JSON)
↓
User-level override (UserPermissionOverride table)
```

Real-time (WebSocket)

`WebsocketModule` exposes a Socket.io gateway. Clients join room `org-{organizationId}`. The server broadcasts:

- `record:created` / `record:updated` / `record:deleted`
- `notification:new`
- `task:updated`

Job Queue (BullMQ + Redis)

Async work (workflow execution, email sending, bulk imports) is offloaded to BullMQ workers. Requires Redis on `REDIS_URL`.

Data Model Philosophy

Metadata-driven Records

```
DynamicModule → defines a "table" (e.g., "Contacts")
├─ Field[] → defines columns (name, type, options)
│   └─ FieldOption[] → enum values for SELECT/RADIO

Record → one row; data stored as JSON blob
└─ data: { fieldName: value, ... }
```

Form System

```
Form
├─ FormSection[] → visual grouping
├─ FormField[] → references module Fields
├─ FormPermission[]
└─ FormSubmission[]
```

`Form.settings` is a freeform JSON column used to store feature configs (OCR, submission receipt, conditional logic) without needing migrations.

Blueprint (Stage Workflow)

```
Blueprint
├─ phases: JSON → named stages
├─ transitions: JSON → allowed moves between stages
└─ BlueprintTask[] → approval tasks per transition
```

Key Design Decisions

Decision	Reason
MySQL + JSON columns	Flexibility for metadata; relations stay normalized
Soft deletes (<code>isDeleted</code>)	Audit trail; records recoverable
Field options in DB	Admin can add options without code deploy
Form settings in JSON	Extend form features without migrations
Portal as separate auth	Customers get their own credential space
Prisma over raw SQL	Type-safety, migrations, easy relation loading
Next.js proxy	No CORS issues in dev; same URL pattern in prod